

Agentic AI in Industry: Adoption Level and Deployment Barriers

Spyridon Alvanakis Apostolou¹, Jan Bosch^{1,2}, and Helena Holmström Olsson³

¹ Chalmers University of Technology, Department of Computer Science and Engineering, Göteborg, Sweden spyalv@chalmers.se

² Eindhoven University of Technology, Department of Mathematics and Computer Science, Eindhoven, Netherlands jan.bosch@chalmers.se

³ Malmö University, Department of Computer Science and Media Technology, Malmö, Sweden helena.holmstrom.olsson@mau.se

Abstract. Agentic AI systems are entering software engineering workflows, yet empirical evidence on how industrial organizations actually adopt them remains sparse. We present a qualitative interview study with sixteen practitioners across twelve companies of varying size and domain. This study characterizes the current agentic AI adoption state of these companies, employing a six-level maturity framework adapted from established AI-driven organizations. The findings reveal that seven companies operate at Level 1 (AI Assistants), four companies at Level 2 (AI Compensators), and only one in Level 3 (Multi-Agent Orchestration), with large and safety-regulated organizations among the most advanced adopters. The primary finding is a capability-deployment verification gap, four companies demonstrated higher-level experimental AI capabilities but cannot integrate them into production workflows because adequate output verification mechanisms are absent, leaving human-in-the-loop as the only trusted verification mechanism. This gap is shaped by four recurring barriers: context window of LLMs constraints especially when diverse knowledge aggregation is needed, under-performance on proprietary programming languages and protocols, non-determinism incompatible with qualification standards, and data confidentiality concerns. Two inter-dependent dimensions of this gap emerge from these findings (information asymmetry and qualification absence) framing a core open problem for industrial agentic integration.

Keywords: Agentic AI · Large Language Models · Software Engineering · Industrial Adoption · Interview Study · Deployment Barriers

1 Introduction

The emergence of generative AI has triggered a paradigm shift in software product development automation, with recent agentic systems demonstrating task-oriented behavior, autonomous decision-making, and self-refinement capabilities, positioning them as the frontier of AI-driven software development. These characteristics push automation boundaries within the Software Development Life Cycle (SDLC)

beyond the "code generation assistant" concept toward autonomous goal-driven systems [7].

Widely adopted assistants such as GitHub Copilot integrate LLMs to amplify developer productivity [11], while newer tools such as Claude Code extend this toward agentic task completion [21]. This transition from assistance to autonomy raises critical questions: where do organizations currently stand on the path toward agentic integration, and what prevents them from advancing further?

Despite the broad interest in AI implementations, studies focusing on industrial adoption remain sparse. To effectively explore industry adoption and address these questions, we conducted an interview study with 16 practitioners across 12 individual companies. We developed a semi-structured and adaptive-type questionnaire, based on the level of implementation of AI tools and systems, aiming to surface meaningful content across organizations at fundamentally different integration stages, unconstrained from a single, unified set of questions.

Our study makes four contributions. First, we provide an empirical characterization of AI integration maturity levels across 12 companies of varying size and domain, revealing that large and safety-regulated organizations are among the most advanced adopters. Second, we identify four recurring barriers that practitioners reported as primary blockers to production deployment, with specific mitigation strategies documented from four companies that demonstrated capabilities beyond their production maturity level. Third, we identify a capability-deployment verification gap characterized along two interdependent dimensions (information asymmetry and qualification absence) that together frame the core open problem for industrial agentic adoption.

The remainder of this paper is organized as follows: Section 2 discusses the gains and limitations of AI tools and systems within the SDLC, as well as the related work. Section 3 details our research methods and analytical approach. Section 4 reports findings across interviews. Section 5 discusses implications of the findings. Section 6 addresses validity threats, and Section 7 concludes and provides future directions.

2 Background and Related Work

2.1 AI in SDLC and Shift to Automation

In the last few years, LLMs have rapidly transformed the software development process, starting initially as assistants for inline code completion and currently heading towards autonomous agents. The most widely deployed tools (e.g., GitHub Copilot, Claude Code, and Codex) integrate into developer workflows through IDE plugins that provide inline suggestions, conversational chat panels for design-level queries, and, increasingly, CLI-based agents for terminal-driven workflows [2]. Studies suggest that these assistants are helping developers reduce the keystrokes and finish tasks faster than before, with 44% often accepting the generated code as it is [11]. Studies in industrial companies show that these assistants also accelerate other processes such as code review, with 26.08% observed increasing in pull requests across large companies, with developer's increased activity [18].

Increased produced code does not directly entail quality and safe code. Assistants, although they increase productivity, seem to lack consideration of parts of the developer intent, such as requirements and security [11, 23]. Additionally, while these tools gain deeper integration in real-world processes, studies show that semantically equivalent but differently worded prompts produce different code outputs, raising robustness concerns. Also, inherent LLM limitations, such as hallucinations in code generation or context rot, as the model searches for a needle in a haystack, hamper the integration of these tools in real-world codebases as they lack developer’s trust [2, 9, 12].

AI is currently deployed across the full range of the SDLC, with large technology-leading companies having already integrated AI tools into their development processes [3]. As LLM capabilities advance and developer productivity follows the same path, projections indicate that AI adoption in software engineering (SE) will boost the automation gains over the next years [10]. Studies demonstrate promising results employing agentic systems on established benchmarks for specific tasks such as fault localization, and end-to-end software generation [13, 15], and industrial workflows where agentic systems reduce task completion time with higher accuracy than AI assistant systems [17]. This indicates automation is shifting from reactive, human-driven processes toward proactive, autonomous AI-driven solutions [10].

2.2 Related Work

There is a need to dive deeper into empirical evidence on how practitioners and organizations are adopting these systems in real-world industrial cases. Hughes et al. provide a broad multi-expert analysis of agentic systems across industries, identifying critical adoption barriers including compatibility with legacy systems, accountability in shared decision-making processes, and the lack of a governance framework for autonomous operations [8].

More focused on SE, two recent qualitative studies align closer to our interview study. Targeting the phases of SDLC, Akbar et al. conducted an interview study with practitioners from industrial companies, academia, and hybrid roles, examining how agentic AI is perceived and used across the SDLC phases [1]. The study findings indicate that implementation is concentrated in code generation and bug detection, while additionally ethical concerns, human skill gaps, and mature tools are identified as wider adoption barriers. Sun and Staron examined agentic pipeline adoption in embedded SE, reporting 11 emerging practices and 14 challenges through a semi-structured interview with senior practitioners from four companies [19]. Their work underlines how safety-critical constraints, such as determinism, traceability, and regulatory compliance, make agentic adoption in embedded systems qualitatively different from general software development. Moreover, Ferino et al. conducted 22 semi-structured interviews with software practitioners, applying socio-technical grounded theory to identify benefits and disadvantages of LLM adoption across individual, team, organisation, and society levels, finding that productivity gains coexist with degraded code quality, skill

atrophy, security and privacy concerns, and reduced mentorship opportunities, with balanced human oversight proposed as the mitigation strategy [5].

These contributions present some common barrier milestones: context window limitations in large code bases, trust issues due to the non-deterministic nature of LLMs, sensitive data confidentiality concerns in cloud-based LLMs, and performance gaps between open-source repositories used in research studies and large, proprietary legacy codebases of real product companies [1, 5, 8, 19].

3 Methodology

3.1 Research Design

We conducted an exploratory qualitative study employing semi-structured interviews, following the guidelines for case study research in SE [16, 20, 22]. The goal of the study is to characterize the state of agentic AI integration in industrial SDLC processes, identify the practical real-world challenges, how the practitioners cope with these, and to understand the barriers that prevent deeper integration of agentic AI systems.

3.2 Maturity Framework as Analytical Lens

The structure of the interviews was developed based on Bosch’s and H. Olsson’s work on AI-driven organizations [4]. In our interview, we defined six levels of agentic AI integration into the SDLC processes, starting from non-organizationally supported assistant frameworks as Level 0 until fully autonomous and self-healing systems as Level 5. Notably, the term of "agentic AI adoption" denotes the full integration trajectory, from non-agentic AI assistants through task-specific agents to fully autonomous systems. We consider agentic behavior capable to begin at Level 1, as this level denotes organizational provision of tools that enable agentic behavior, not that all usage patterns within that level are agentic. The description of the maturity levels in Table 1.

The adopted framework [4] is an inductively derived maturity model, which characterizes how organizations evolve toward AI-driven development through five progressive stages. Since this model was itself derived from practitioner interviews, its level definitions carry empirical grounding consistent with our study’s qualitative methodology. We adopted the level structure without modification to its definitions, applying it with a dual target. First, as an adaptive routing mechanism during interviews to determine the level-specific question set, enabling context-specific conversation rather than a single generic set attempting to fit diverse AI integration levels. Second, as a systematic post-hoc analytical lens, supporting cross-company comparison within and across maturity levels.

3.3 Participant Selection

The participants were recruited based on their years of experience in SE in an industrial concept, targeting practitioners whose daily tasks align directly with

a part of SDLC processes. All of them consented in advance to participate the study anonymously. The transcripts of the interviews are accessible only to the research team.

To assure diversity across the companies size, application domain, regulatory environment, and other parameters that are expected to influence the AI integration in SDLC, we implemented the purposive sampling as proposed by Patton et al. [14]. We conducted sixteen interviews with participants from twelve individual companies. These are separated into three groups based on the size: small (<100 employees), medium (100–1000 employees), and large (>1000 employees). Our sample includes three small companies (C1, C2, C3), two medium (C4, C5), and seven large companies (C6-C12) as described in Table 2 along with the participant’s years of experience in SE and their job positions.

Table 1. Agentic AI Maturity Levels

Lvl	Name	Description
0	Individual Use	Personal AI usage without formal organizational backing
1	AI Assistants	LLM tools enhancing productivity (e.g., GitHub Copilot)
2	Task Agents	AI agents handle specific roles/tasks (e.g., code review)
3	Collaborative AI	Human-managed multi-agent workflows across SDLC phases
4	System Builders	Autonomous generation of systems from high-level intent
5	Self-Optimizing AI	Systems that self-heal and regenerate without human input

3.4 Interview Protocol

The interviews have 4 sequential components and have an average duration exceeding 45 minutes. The components are:

1. **Demographics & Context:** Role, experience, product domain, regulatory environment, and self-assessed familiarity with agentic AI concepts.
2. **Maturity Assessment:** An assessment to classify the organization’s current level of AI tools integration, as described in Table 1.
3. **Level-Specific Deep Dive:** A set of questions tailored to the identified maturity level, exploring workflow impact, trust calibration, integration challenges, legacy code handling, and unmet needs.
4. **Future Outlook & Research Alignment:** Practitioner perspectives on where agentic AI would be most impactful, their company’s internal data characteristics, and preferred agent roles (supervisor vs. worker).

Table 2. Participant and Company Overview

Comp.	Size	Domain	Part. ID	YoE	Position
C1	Small	Maritime Software	P1	4	Head of Research
C2	Small	Data Analytics	P2	8	Product Owner
C3	Small	Data Analytics	P3	5	AI Engineer
C4	Medium	Fintech	P4	10	Soft. FE Eng.
C5	Medium	Automotive Safety	P5	10	Tech. Exp. in SC Dev.
C6	Large	Automotive	P6	25	Senior Technical Leader
			P7	10	Res. in AV
			P8	25	Tech. Ldr in TDD
C7	Large	Telecommunications	P9	26	Lead Soft. Arc.
			P10	19	Spc Data Man. & Sec
C8	Large	Manufacturing	P11	2	Emb. Soft. Dev.
			P12	12	Lead Engineer
C9	Large	Manufacturing	P13	30+	Research Manager
C10	Large	Consulting	P14	2	Data Scientist
C11	Large	Consumer Goods	P15	2	Data Analyst
C12	Large	Pharmaceutical	P16	5	Data Anon. Spc

3.5 Data Analysis

The transcripts were analyzed following a cross-case synthesis approach [22]. Each interview was individually reviewed and summarized in a structured text that follows the same four structure parts as the interview schema: demographic fields (*Job Title*, *Main Tasks*, *Software Type*, *Regulations*), the assigned *AI Level* and tools in active use, *Notes* on workflow observations and experimental activities deriving from the level-specific question set, and *Limitations* and *Needs* capturing reported barriers and unmet requirements.

To mitigate information loss during summarization and to assure data privacy, we independently queried two local LLM models (gpt-oss-20b from OpenAI and Qwen3-14B), providing in each query a raw interview transcript with the corresponding structured summary and a descriptive one-shot prompt to surface any missing details (while referring to the corresponding interview text for easier evaluation) from the researcher’s summary. Across the 16 summaries, the two models collectively surfaced 62 suggestions, of which 11 were accepted after manual verification. The accepted additions concerned contextual details rather than core elements. The augmented summaries were then systematically compared across cases following the comparison approach of Glaser and Strauss, reported limitations were iteratively grouped, re-assessed across multiple passes

and progressively unified into higher-order barrier categories until the groupings stabilized [6].

4 Findings

4.1 Maturity Assessment

Of the 12 companies interviewed, 7 operate at Level 1 (AI Assistants) regarding the maturity assessment of AI integration, 4 at Level 2 (AI Compensators), and 1 at Level 3 (AI Superchargers). No company reported operating at Levels 0, 4, or 5. This distribution, demonstrated in Table 3, reveals significant clustering at the "AI assistants" level, while interestingly, companies at the "AI compensator" level are predominantly large organizations or restricted by safety regulations. Company C3 is the only company that reaches the "AI superchargers" level in production mode.

Table 3. Maturity level distribution by regulatory status

Companies	Level 1	Level 2	Level 3
Without regulations	2	1	1
With regulations	5	3	–
Total	7	4	1

Table 3 also demonstrates which companies are restricted by security or safety regulatory constraints. Companies C5 and C6 are restricted by safety regulation constraints, while C7's constraints depend on each country's legislation. Other companies (C8, C9, C11, C12) are similarly affected by regulations, and in many cases (specifically for C11 and C9) regulations vary between projects and countries. Interestingly, Table 3 demonstrates that despite regulations (safety or security) and legacy code management challenges, organizations at the "AI Compensator" level are not small, flexible, or unrestricted companies. This indicates that while regulatory constraints create additional barriers, they do not determine the AI adoption outcome, rather successful progression depends on strategic investment in integration infrastructure and organizational commitment to sustained deployment efforts.

It is worth noting that companies C6, C7, C8, and C12 stated during the interviews that they are actively experimenting with agentic integrations corresponding to the next maturity level beyond their current workflow integration level.

4.2 Level-specific Insights

Level 1: The AI Assistant Plateau Seven companies (C1, C2, C8, C9, C10, C11, and C12) operate at the "AI Assistants" level. Across these companies, the

assistants are most actively used in code generation, debugging, documentation drafting, and code explanation. Productivity gains were consistently reported, with some individual developers mentioning that tasks such as scripting and testing have improved by an order of magnitude.

Despite these gains, limitations hamper seamless progress to the next maturity level. First, inherent LLM limitations such as hallucinations and non-deterministic behavior lead developers to mistrust their outputs (especially in embedded systems). Participants P11 and P12 noted that complex or niche tasks require more time to understand and debug the generated output than to write it from scratch, while participant P1 mentioned silent model version changes produce inconsistent agent behavior, making reliable use difficult even within the same project. Second, industrial codebases may contain millions of lines of code and thousands of relevant documents in diverse locations, exceeding the context window of models and hindering effective use without a context management layer. Third, developers particularly in companies with sensitive data management, remain cautious with cloud LLM models due to data leakage concerns regardless of enterprise agreements for data confidentiality, leading some of them to employ personal and local LLM solutions rather than company-approved tools.

A recurring observation across Level 1 companies is that AI usage is mostly individual: adoption patterns, prompt strategies, and tool selection are left to each developer, with limited organizational guidance. Notably, the degree of agentic utilization also varies per individual, assistants may be used as simple chatbots, context-aware code completion tools, or lightweight agentic workflows where the provider supports it (e.g., GitHub Copilot). Interestingly, two practitioners from C8 and C12 reported active experimentation toward task-specific agents, as described in Section 4.3.

Level 2: Task Ownership and Trust Calibration Four companies (C4, C5, C6, and C7) have progressed to deploying AI agents with defined task ownership within their SDLC. The agent-owned tasks predominantly include documentation generation, pull request analysis, code review, and log or fault analysis. These are well-scoped, lower-risk activities rather than core implementations and, notably, are concentrated in the post-development phases of the SDLC.

Human verification is an essential and consistently reported requirement across Level 2 deployments. Agents are not trusted to autonomously complete a pipeline stage, rather their outputs serve as structured inputs to a human evaluator. Practitioners described trust calibration as a necessity, with agents deployed in cases where failure is auditable and where guardrails or manual gatekeeping can be applied at safety- or quality-critical stages, with participant P4 mentioning that integrated agentic systems tend to overengineer problems generating unnecessary code. Companies operating under safety-critical regulations or managing large legacy codebases with proprietary languages (C5, C6, and C7) reported additional caution, implementing formal testing layers and multi-reviewer processes as guardrails around agent outputs.

Participants from C5, C6, and C7 reported performance degradation in company-specific programming languages and the necessity of effective context management strategies for their large codebases. RAG-based knowledge layers were reported to partially mitigate the performance gap but did not fully close. Context management in terms of retrieving accurate information without exceeding the context window is identified as a primary technical bottleneck for further adoption.

For large organizations such as C6 and C7, context management does not refer solely to department-specific information but to multidisciplinary concepts requiring fragmented input from diverse parts of the company. Both organizations are actively experimenting with multi-agent orchestration corresponding to Level 3 (as described in Section 4.3), though these remain outside active production deployment.

Level 3: Multi-Agent Orchestration at the Boundary One small analytics and software services company (C3) is classified at Level 3 based on deployed multi-agent pipelines in which agents delegate tasks and pass outputs to one another autonomously. Specifically, this includes a task-delegation agent that routes work to specialized downstream agents and a ranking agent that re-evaluates the relevance of retrieved context in RAG-based client deployments.

This company presents a boundary case. The Level 3 classification reflects client-facing deployments built on a cloud platform that provides native tooling for agent composition and inter-agent coordination and infrastructure that smaller, project-based companies without legacy systems can adopt more easily than larger organizations. Reported limitations are consistent with those at lower levels: performance degradation in longer agent conversations can allow low-level incorrect solutions to propagate through the pipeline, and human semantic review of pull requests remains necessary.

4.3 Capability without Deployability

Four companies (C6, C7, C8, and C12) demonstrated a gap between experimental agentic capabilities and their qualification for integration into active development workflows, what we refer to throughout this paper as the capability-deployment verification gap. It is worth noting that the agentic systems discussed here function as workflow tools. The barrier described, is one of integration into the development process, not the product deployment in the traditional sense.

Among Level 1 companies, C8 has developed an internal agent-assisted knowledge tool that retrieves information from diverse and fragmented documentation sources on developer demand. Python-only AI tools were developed, as AI-assisted embedded code generation is explicitly avoided in production due to the difficulty of fully reviewing large generated code snippets. In addition, the lack of compiler-aware debugging context that a human developer naturally possesses makes debugging difficult to achieve by agents alone, or as participant P12 stated:

"It is an unfair comparison between the developer and the agents, since the agents do not have all the information".

Similarly, company C12 is in early experimental stages of task-specific agentic integration into code review processes, extending assistant usage toward task ownership automation. No workflow-integrated agent deployment has occurred yet, as the tooling is still under evaluation and integration into existing codebases.

Among Level 2 companies, C6 and C7 reported more advanced experimental stages. On the other hand, C7 multi-agent workflows are already operational within the copilot environment, reducing bug resolution turnaround from days (or weeks) to hours. Yet, agentic end-to-end pipelines remain absent from active development workflows due to several barriers:

1. Massive documentation across diverse sources makes context window management a limiting factor.
2. Non-determinism of LLM outputs, for which participant P9 mentioned that the same prompt may yield substantially different code depending on the model version, introducing reproducibility and traceability concerns. A concrete manifestation of this is global team coordination, when teams working on the same codebase from different geographical regions may receive different model versions, resulting in conflicting AI-generated outputs.
3. The need to reproduce previous states of older versions extends to a contractual requirement that cloud LLM vendors provide model continuity, including model handover in cases of vendor bankruptcy.
4. Proprietary languages and hardware-specific toolchains for which no reliable LLM performance baseline exists.
5. Data confidentiality constraints restrict the use of cloud-based LLMs for sensitive internal codebases.

These barriers are partially addressed. C7 separates local and cloud LLM usage based on data sensitivity, mitigating confidentiality exposure and versioning management issues. In addition, large contexts are batched into sub-contexts to work within window constraints, and model knowledge is supplemented with internal documentation.

At C6, safety-critical regulations in automotive development impose a qualification requirement on any tool integrated into the SDLC, since tools must be traceable and predictable, meaning that their behavior boundaries must be documented and validated before use. Practitioner P7 articulated this requirement precisely:

“In safety-related fields, if you want to use a tool you need to know what are the bugs or the weaknesses in that tool to be able to use it in your daily task.”

LLMs fail to meet these requirements, as they are non-deterministic by nature. A concrete example of how C6 addresses these constraints is an experimental multi-agent Risk Assessment pipeline: given a function description and a set of malfunctions, the system autonomously decomposes the task into scenario permutations, delegates sub-tasks for severity assessment to specialized agents, and produces a structured requirements table, with a human evaluator positioned at the end of the loop to avoid introducing bias into the assessment chain. Other agentic concepts are explored across the SDLC: log analysis tools, task ticket

classification agents, and multi-agent frameworks for pull request review and test generation operate at the experimental boundary. Context window management remains a consistent limitation, in this case leading to extensive AI tool setup in large codebases, requiring significant effort while remaining not easily reusable across contexts. Mitigation strategies include RAG pipelines combining with graph-based retrieval for legacy context and layered static analysis guardrails (e.g., SonarQube, PoliSpace, and Google Tests) that constrain the error surface of AI-generated code to auditable bounds.

Beyond the aforementioned limitations, legacy tool-chain workflows were developed before the rise of LLM integration, constituting retrofitting constraints as not all tools evolve their AI-supported capabilities at the same rate. Company C5 noted that it operates with Gerrit instead of GitHub, which is not as agent-integrated as the latter. While Gerrit offers properties that justify its utilization in C5’s safety-critical workflows, its distance from modern AI tooling represents an infrastructural barrier for integrating agentic capabilities into the development flow.

Table 4. Cross-case barrier distribution

Comp. (Lvl)	Context	Propriet.	Non-Det.	Data Conf.
C1 (L1)	•		•	•
C2 (L1)			•	•
C8 (L1)	•	•	•	•
C9 (L1)	•		•	
C10 (L1)	•		•	
C11 (L1)	•		•	
C12 (L1)	•	•	•	
C4 (L2)	•		•	
C5 (L2)	•	•	•	
C6 (L2)	•	•	•	•
C7 (L2)	•	•	•	•
C3 (L3)	•		•	

All Level 2 companies noted that human-in-the-loop (at least for verification) is a necessity. Although human review does not scale as generated output volume increases. Company C5 has a two-evaluator requirement for code reviews, as it is a safety-critical requirement to assure quality and bug minimization. Participant P5 noted that the goal is not to replace humans in this process but rather to augment them by reducing the cognitive load without eroding the quality gatekeeping that regulation requires. The scalability problem extends beyond review workload, participant P8 mentioned that many tasks already exceed human tracking capacity, as no human can maintain coherent awareness of 100,000 Jira tickets, fragmented documentation spread across organization departments, or full regulatory traceability of a complex system (such as automotive).

4.4 Cross-cutting Barriers

Four barriers surfaced organically across different question sets and recurrently noted as limitations for agentic adoption in the SDLC phases:

1. **Context management in large industrial input:** Industrial codebases and their associated documents, (e.g., protocols, regulations, etc.), collectively exceed the model’s information capacity. Participants from C6 noted that a RAG-based layer over internal information mitigated but not resolve the problem, as RAG works great when the documentation is well structured and the questions are straightforward, but fails for complex cross-cutting queries. In addition a combination of RAG and graph-based retrieval improved precision, but yet the approach remained context-scoped and non-reusable across systems. Participants from C7, noted that a workaround solution is batching large contexts into chunks of a vector database to work within capacity constraints, yet the maintenance and updating of this database is costly and does not scale well as the codebase evolves.
2. **Underperformance in proprietary content:** It is reported that in domain-specific programming languages, hardware-specific tool chains, and company-specific protocols AI tools underperform as models are not trained in them. Participants from companies C5 and C6 mentioned that RAG-based knowledge layers, LoRA fine tuning or even prompt injection with detailed documentation mitigated the problem of underperformance, but still did not meet company’s quality criteria. Notably, participant P9 mentioned that in some cases they are training their own models from scratch for internal proprietary languages, although this is costly and limited.
3. **Non-determinism and qualification incompatibility:** Probabilistic output generation makes LLM behavior difficult to bound, creating qualification challenges, especially in safety-critical companies. Participant P8, noted the usage of tools such as SonarQube, Google Tests, and PoliSpace as a static guardrail layer. Despite the mitigation of the issue, these tools operate mostly as code-quality assurance and they lack capabilities to verify semantic correctness, cross-cutting regulatory intent, or conformance to requirements distributed across standards documents and architectural specifications.
4. **Data confidentiality:** Practitioners do not trust cloud LLM providers with sensitive internal code and data. This limitation is effectively addressed by C7, using sandboxed environments with constrained models with no external connectivity, having as cost of confidentiality the reduction of capabilities that the most capable cloud-hosted models provide.

These four barriers reflect on the same time *structural* and *functional* limitations of LLMs. Focusing solely on the functional aspect, we distinguish the first three barriers, as they collectively describe a system that lacks sufficient information access to perform reliably and lacks verification mechanisms to bound its unreliable behavior. These limitations fall within the scope of agentic AI integration in the SDLC, as they concern how agents access, process, and qualify

information throughout development workflows. Data confidentiality, while consistently reported, is a deployment infrastructure constraint addressed through sandboxed environments, local models, and federated architectures, areas that fall outside the scope of agentic system. Therefore we focus the following discussion only on the functional aspects of the first three barriers and the research directions they motivate.

5 Discussion

Level 1 companies provide enterprise-licensed AI tools as an organizational baseline, reflecting increased AI presence in professional workflows. Notably, productivity gains are not uniform across SDLC phases, as participant P2 noted that AI tool usage has become almost necessary in earlier phases because departments engaging with later stages have become significantly more productive, creating flow pressure that extends the identified barriers beyond code-connected tasks alone. The provision of AI tools enhances experimentation while exposing each individual to limitations that are not resolvable at the individual level. At Level 2, companies move beyond tool provision toward task-specific automation, shifting from individual productivity enhancement to organizational process integration. Companies at both levels reported limitations as described in Sections 4.3 and 4.4, manifesting differently according to the scope of utilization but meet on the same barriers, which are not resolvable through model capacity improvements alone.

As documented in Section 4.3, industrial practitioners are not waiting for more capable systems but for methods to sufficiently inform and verify the systems they are already building in order to meet production qualification requirements. This capability-deployment verification gap manifests across two core dimensions.

The first dimension is *information asymmetry*. Effective task delegation requires coherent access to architectural, regulatory, and functional information that currently lives in fragmented locations across organizational departments. As participant P12 articulated, the comparison between a developer and an agent is unfair because agents do not have access to the same information. The challenge, is not only the volume of the information but also the heterogeneity, meaning that codebases, regulatory documents, requirements, Jira tickets, and architectural specifications differ in structure, quality, and format, making information retrieval much harder than single-source context injection.

The second dimension is *qualification absence*. Particularly in safety-regulated companies, any tool integrated into the SDLC must have documented behavior and weaknesses, which the LLMs non-deterministic nature do not satisfy as requirement. Existing guardrails (e.g., static analysis, test suites) address code quality but do not verify semantic correctness or alignment with cross-cutting requirements distributed across documents and architectural specifications. Human-in-the-loop remains the only trusted verification mechanism, yet it does not scale as generated output volume increases, and many tasks already exceed human tracking capacity.

The findings of Table 4 support this two-dimensional structure. Context management limitations were reported by 11 of 12 companies, with only exception company C2. Participant P2 noted that the AI assistants usage mostly focuses on brainstorming, research, and requirements documentation, with all the information provided to the tools and no cross-referencing or fragmented documentation. Additionally, all the companies operating with proprietary languages or protocols (C5, C6, C7, C8, C12) reported underperformance as barrier, reflecting both an information gap (models lack training exposure) and a qualification concern (outputs in unfamiliar languages are harder to verify). Finally, non-determinism as a limitation, aligns strongly with the qualification dimension, as different model versions, silent version updates and prompt sensitivity, produces inconsistent results that later requires extended human evaluation for qualification assurance.

C3 was the only company that reached Level 3 agentic AI integration in production deployment, not because it exceeds the others in technical sophistication, but rather because it operates in a lower-risk analytic domain without formal qualification requirements on a cloud platform that natively supports agent composition. In this case, neither the information asymmetry nor the qualification requirement applies at the same scale, and the infrastructure was built with AI-native tooling from the outset.

Relation to Prior Work: All four barriers identified in our study have partial precedent in prior work. Sun and Staron [19] reported certification, non-determinism, and organizational silos as embedded-specific constraints, and Akbar et al. [1] identified the absence of evaluation standards and agent-aware infrastructure as blockers.

Our contribution is not the individual identification of these barriers but the empirical observation that they jointly constitute a capability-deployment verification gap. Additionally, we document practitioner mitigation strategies from companies experimentally ahead of their production maturity level, revealing not only what blocks adoption but how organizations partially circumvent it. Ultimately, practitioners possess the agentic capability, yet lack mechanisms to qualify outputs for production, leaving human-in-the-loop as the only trusted but non-scalable mechanism.

6 Threats to Validity

Internal validity. Maturity assessments rely on self-reporting rather than direct observation. The adaptive interview design mitigated this by routing follow-ups based on claimed maturity against specific implementation details, however, companies at different levels received partially different question sets, so cross-level comparisons rely on organically surfaced themes rather than uniform elicitation.

Construct validity. The maturity framework was adapted from established work [4], providing definitional grounding. Companies experimenting beyond their production level were resolved by explicitly distinguishing experimental capability from production deployment throughout the analysis.

External validity. Recruiting through professional networks introduced self-selection bias toward highly AI-aware practitioners, so findings cannot be generalized beyond organizations already actively adopting agentic AI.

7 Conclusion & Future Directions

In this study, employing a six-level maturity framework as an analytical lens, we made an empirical characterization of agentic AI adoption across 12 individual companies. The majority of the companies operate at Level 1 (AI Assistants), with four companies at Level 2 (AI Compensators) and one at Level 3 (Multi-Agent Orchestration). Notably, the most advanced companies are large or safety-regulated organizations, indicating that organizational investment outweighs regulatory or size constraints as progression factors.

The main finding is a capability-deployment verification gap, four companies demonstrated experimental agentic capabilities beyond their production maturity level, but they cannot integrate them into production development, as methods that sufficiently verify agentic outputs against industrial qualification requirements are absent. Leaving human-in-the-loop as the only available verification mechanism, although humans do not scale as generated output volume increases. In addition, we mapped four recurrently addressed barriers that hinder closing the capability-deployment verification gap, from which future research directions are shaping as unresolved problems.

Future Directions: The two identified dimensions are not independent, but together they define a capability-deployment verification gap that constrains industrial agentic adoption. Combining them, research should be steered towards verification methodologies that account for intent alignment, maintainability constraints, and architectural dependencies across diverse information sources. Translating these dimensions into verification mechanisms will not eliminate the error space of a probabilistic system, but it can narrow it significantly while extending the context scope of an agentic system during a task execution, addressing this way both dimensions in a unified approach.

References

1. Akbar, M.A., Khan, A.A., Hamza, M., et al.: Agentic AI in Software Engineering: Practitioner Perspectives Across the Software Development Life Cycle (Sep 2025), <https://papers.ssrn.com/abstract=5520159>
2. Akhoroz, M., Yildirim, C.: Conversational AI as a Coding Assistant: Understanding Programmers' Interactions with and Expectations from Large Language Models for Coding (Mar 2025), <http://arxiv.org/abs/2503.16508>
3. Alenezi, M., Akour, M.: AI-Driven Innovations in Software Engineering: A Review of Current Practices and Future Directions. *Applied Sciences* **15**(3) (Jan 2025), <https://www.mdpi.com/2076-3417/15/3/1344>
4. Bosch, J., Olsson, H.H.: Towards AI-Driven Organizations. In: *Software Engineering and Advanced Applications*. pp. 280–295. Springer Nature Switzerland, Cham (2026). https://doi.org/10.1007/978-3-032-04207-1_19

5. Ferino, S., Hoda, R., Grundy, J., Treude, C.: Walking the Tightrope of LLMs for Software Development: A Practitioners' Perspective (Nov 2025), <http://arxiv.org/abs/2511.06428>, arXiv:2511.06428 [cs]
6. Glaser, B., Strauss, A.: Discovery of grounded theory: Strategies for qualitative research. Routledge (2017)
7. He, J., Treude, C., Lo, D.: LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision and the Road Ahead (Apr 2024), <https://arxiv.org/abs/2404.04834v4>
8. Hughes, L., Dwivedi, Y.K., Malik, T., et al.: AI Agents and Agentic Systems: A Multi-Expert Analysis. *Journal of Computer Information Systems* **65**(4), 489–517 (Jul 2025), <https://doi.org/10.1080/08874417.2025.2483832>
9. Kelly, H., Anton, T., Jeff, H.: Context Rot: How Increasing Input Tokens Impacts LLM Performance (Jul 2025), <https://research.trychroma.com/context-rot>
10. Kwa, T., West, B., Becker, J., et al.: Measuring AI Ability to Complete Long Tasks (Mar 2025), <http://arxiv.org/abs/2503.14499>
11. Liang, J.T., Yang, C., Myers, B.A.: A Large-Scale Survey on the Usability of AI Programming Assistants: Successes and Challenges (Sep 2023), <http://arxiv.org/abs/2303.17125>
12. Mastropaolo, A., Pascarella, L., Guglielmi, E., et al.: On the Robustness of Code Generation Techniques: An Empirical Study on GitHub Copilot. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). pp. 2149–2160 (May 2023), <https://ieeexplore.ieee.org/abstract/document/10172792>
13. Nguyen, M.H., Chau, T.P., Nguyen, P.X., et al.: AgileCoder: Dynamic Collaborative Agents for Software Development based on Agile Methodology (Jul 2024), <http://arxiv.org/abs/2406.11912>
14. Patton, M.Q.: Qualitative Research & Evaluation Methods: Integrating Theory and Practice. SAGE Publications (Oct 2014)
15. Qin, Y., Wang, S., Lou, Y., et al.: SoapFL: A Standard Operating Procedure for LLM-Based Method-Level Fault Localization. *IEEE Transactions on Software Engineering* **51**(4), 1173–1187 (Apr 2025), <https://ieeexplore.ieee.org/document/10891926>
16. Runeson, P., Höst, M.: Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering* **14**(2), 131–164 (Apr 2009), <https://doi.org/10.1007/s10664-008-9102-8>
17. Sawant, P.: Agentic AI: A Quantitative Analysis of Performance and Applications (Feb 2025), <https://www.preprints.org/manuscript/202502.1647>
18. Stray, V., Brandtzæg, E.G., Wivestad, V.T., et al.: Developer Productivity With and Without GitHub Copilot: A Longitudinal Mixed-Methods Case Study (Jan 2026), <http://arxiv.org/abs/2509.20353>
19. Sun, S., Staron, M.: Agentic Pipelines in Embedded Software Engineering: Emerging Practices and Challenges (Jan 2026), <http://arxiv.org/abs/2601.10220>
20. Walsham, G.: Interpretive case studies in IS research: nature and method. *European Journal of Information Systems* **4**(2), 74–81 (May 1995), <https://doi.org/10.1057/ejis.1995.9>
21. Watanabe, M., Li, H., Kashiwa, Y., et al.: On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub (Sep 2025), <https://arxiv.org/abs/2509.14745v3>
22. Yin, R.K.: Case Study Research: Design and Methods. SAGE (2009)
23. Yujia, F., Peng, L., Amjed, T., et al.: Security Weaknesses of Copilot-Generated Code in GitHub Projects: An Empirical Study. *ACM Transactions on Software Engineering and Methodology* (2025), <https://dl.acm.org/doi/10.1145/3716848>